

Campus Event Manager

Project Report - NoSQL Database (MongoDB)

Author: Ahmed ABIDHIAF

Github repository : <https://github.com/abidhiah/campus-event-manager.git>

DB: MongoDB (campus_events)

Stack: Python / Flask / PyMongo

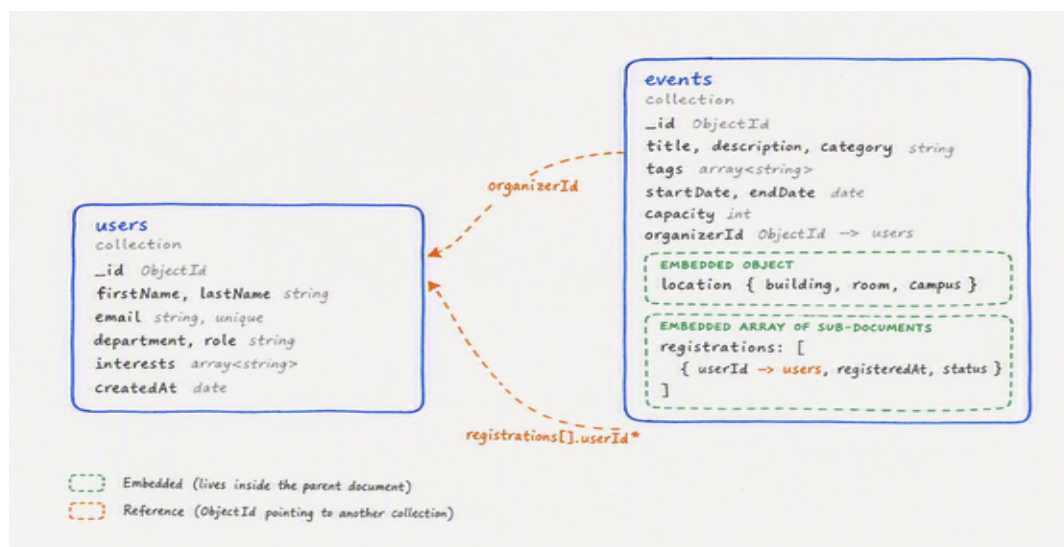
1. Project Overview

The brief asks for a document database designed from scratch, populated with coherent data, and a small application that actually uses it - not a folder of standalone Mongo scripts. I kept the proposed scenario as it was given: a university with no database or application to speak of, needing something lightweight to run its campus events (workshops, talks, hackathons, meetups). The result is called Campus Event Manager.

On the technical side I went with Python, Flask and PyMongo rather than Node or Java. Two reasons for that: Python was already set up on my machine, and more importantly PyMongo lets every query be written out directly instead of hiding it behind an ODM. Since the brief is explicit that queries and pipelines need to stay identifiable in the code, that pushed me away from something like Mongoose. MongoDB runs locally on my Mac (installed through Homebrew), and the interface is server-rendered Jinja2 templates with Bootstrap - nothing fancy, which matches what the brief asks for: this is a NoSQL project, not a UI competition.

2. NoSQL Data Model

The database is called `campus_events` and holds two main collections, `users` and `events`. I didn't add a third one: with only two genuinely distinct entities (people and events), another collection would have been artificial rather than useful.



Collection users

- Each document holds `_id`, `firstName`, `lastName`, `email`, `department`, `role`, `interests` (an array of strings) and `createdAt`. Nothing is embedded here - a user is a simple, self-contained entity.

Collection events

- This is the more interesting collection from a modeling standpoint. A (shortened) document looks like this:

```
{
  title: "Student Robotics Meetup",
  category: "Meetup", tags: ["robotics", "opensource"],
  startDate: ISODate(...), endDate: ISODate(...), capacity: 10,
  location: { building: "Engineering Building", room: "Workshop 3", campus: "Paris" },
  organizerId: ObjectId("..."),
  registrations: [
    { userId: ObjectId("..."), registeredAt: ISODate(...), status: "confirmed" },
    ...
  ]
}
```

3. Modeling Choices: Embedding vs. Reference

The brief explicitly asks for at least two justified decisions, so here are the two that mattered most while building the model.

location is embedded. A venue doesn't mean anything outside of the event it belongs to - nobody is ever going to query "every document held in the Innovation Center" as a standalone entity, and it isn't shared with anything else. Embedding it avoids a pointless join for an object that only has three fields.

registrations is an array of sub-documents, but the userId inside each entry stays a reference. This is the most consequential decision in the whole model. I embedded registrations directly in the event because the event detail page constantly needs to read and update the participant list (count confirmed sign-ups, check that a user isn't already registered, add or remove one entry) and the volume stays small - a few dozen participants per event at most, so there's no risk of hitting the document size limit. What I did *not* do is copy the user's name or email into each registration: I only keep their userId as a reference. A user exists independently of any event (their email can change, and they can be an organizer of one event while attending ten others), so duplicating their info inside every registration would have created an inconsistency to manage every time a profile changes. The same reasoning applies to organizerId on the event: reference, not embedding.

4. Application Architecture

The repository follows roughly the layout suggested in the brief:

```
database/init.py      # collections, schema validation, indexes
database/seed.py      # reproducible seed dataset
src/db.py             # MongoClient connection (via .env)
src/app.py            # Flask app factory, registers the 4 blueprints
src/routes/           # dashboard.py, events.py, users.py, analytics.py
src/templates/        # one Jinja2 view per page
src/static/           # small stylesheet on top of Bootstrap
tests/                # pytest suite for form validation.github/workflows/      # CI:
runs pytest on every push
```

The MongoDB connection goes through `src/db.py`, which reads `MONGO_URI` and `MONGO_DB_NAME` from a `.env` file (git-ignored, with a `.env.example` next to it showing the expected format). Every page is its own

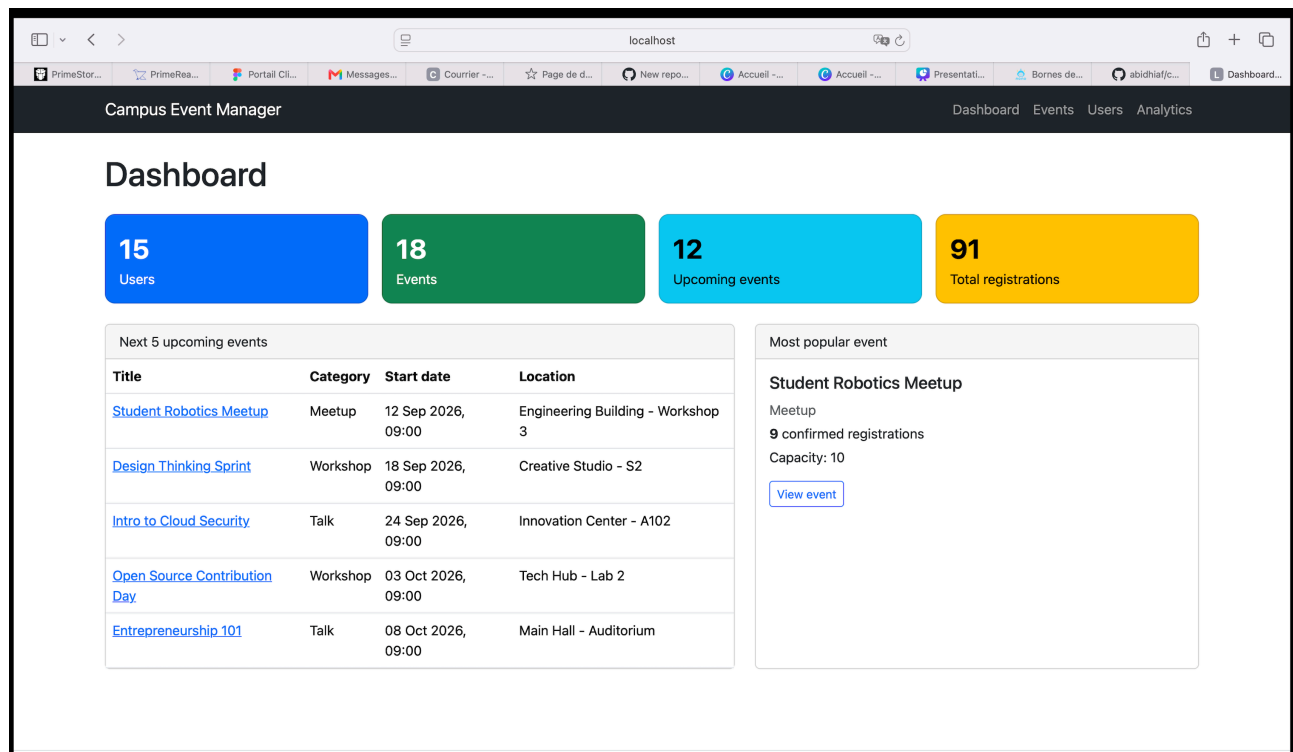
Flask blueprint, so all the MongoDB operations for a given page live in a single file that's easy to find, instead of being spread across a generic DAO layer.

Two smaller additions worth mentioning here: the Events and Users list queries use an explicit `projection` so they only pull the fields the table actually renders, and there are two compound indexes (`category+startDate`, `department+role`) that match the combined filters the UI actually runs, on top of the three mandatory single-field indexes. A GitHub Actions workflow also runs the pytest suite on every push, so a broken validation rule would show up immediately instead of being caught by accident.

5. Application Pages

Dashboard

The home page shows four counters (users, events, upcoming events, total registrations), the next 5 upcoming events, and the most popular event. Nothing is hard-coded: user and event counts come from `count_documents`, the registration total from a `$unwind + $count` pipeline, and the most popular event from a `$filter + $size` on the registrations array, counting only the confirmed status.



Events

Full catalog with search by title, filters by category, tag and time period (past/upcoming), sorting by date, and full CRUD (create, edit, delete). A "Full" badge shows up as soon as confirmed registrations reach capacity. The create/edit form handles the embedded location object and the tags array directly, and the listing query now uses an explicit projection to only pull the fields it renders.

Events

+ New event

Search by title

All categories

All tags

Upcoming & past

Date: soonest first

Filter

Title	Category	Start date	Location	Capacity	Confirmed		
Python for Automation	Workshop	20 Jul 2026, 09:00	Innovation Center - B204	22	3	Past	EditDelete
Career Fair: Tech & Data	Career Fair	27 Jul 2026, 09:00	Main Hall - Auditorium	80	0	Past	EditDelete
Intro to Python for Data Science	Workshop	15 Aug 2026, 09:00	Innovation Center - B204	25	7	Past	EditDelete
Cloud Architecture Fundamentals	Talk	22 Aug 2026, 09:00	Innovation Center - A101	40	3	Past	EditDelete
Campus Hackathon: Smart Mobility	Hackathon	05 Sep 2026, 09:00	Tech Hub - Lab 1	30	0	Past	EditDelete
Cybersecurity Career Night	Career Fair	09 Sep 2026, 09:00	Main Hall - Auditorium	60	7	Past	EditDelete
Student Robotics Meetup	Meetup	12 Sep 2026, 09:00	Engineering Building - Workshop 3	10	9	Open	EditDelete
Design Thinking Sprint	Workshop	18 Sep 2026, 09:00	Creative Studio - S2	20	3	Open	EditDelete
Intro to Cloud Security	Talk	24 Sep 2026, 09:00	Innovation Center - A102	35	4	Open	EditDelete
Open Source Contribution Day	Workshop	03 Oct 2026, 09:00	Tech Hub - Lab 2	25	4	Open	EditDelete
Entrepreneurship 101	Talk	08 Oct 2026, 09:00	Main Hall - Auditorium	50	5	Open	EditDelete
AI in Healthcare Panel	Talk	14 Oct 2026, 09:00	Innovation Center - A101	45	0	Open	EditDelete
Networking Night with Alumni	Meetup	20 Oct 2026, 09:00	Main Hall - Terrace	60	5	Open	EditDelete

Event Detail and Registrations

This is where registrations actually get managed: pick a user, register them (\$addToSet keeps duplicates out), refuse a confirmed registration once the event is full (the user is placed on the waiting list instead), and remove a registration with \$pull. The participant list and occupancy percentage update immediately.

Student Robotics Meetup

EditDelete

Meetup

Monthly meetup for the robotics club to share ongoing projects.

robotics

opensource

Start / End

12 Sep 2026, 09:00 → 12 Sep 2026, 11:00

Location

Engineering Building, room Workshop 3 - Paris campus

Organizer

Emma Rousseau

Capacity

9 / 10 (90.0% occupancy)

Register a user

Select a user

Register

Participants (14)

Name	Status	
Alice Martin	Confirmed	Remove
David Nguyen	Waiting	Remove
Lucas Bernard	Waiting	Remove
Sofia Costa	Confirmed	Remove
Fatou Diallo	Confirmed	Remove
Yanis Benali	Waiting	Remove
Marco Rinaldi	Cancelled	Remove
Lea Moreau	Cancelled	Remove
Claire Petit	Confirmed	Remove
Nora El Amrani	Confirmed	Remove

Users

User directory with search by name/email, filters by department and role, and each user's registration count, computed with an `$unwind + $group` pipeline over the events collection. Deleting a user is blocked while they're still referenced as an organizer or in a registration, so no `ObjectId` is ever left dangling.

Campus Event Manager

DashboardEventsUsersAnalytics

Users

+ New user

Search by name or email

All departments

All roles

Filter

Name	Email	Department	Role	Interests	Registrations	
Yanis Benali	yanis.benali@campus.fr	Computer Science	student	cybersecurityopen source	5	Edit
Lucas Bernard	lucas.bernard@campus.fr	Mechanical Engineering	student	roboticssports	7	Edit
Sofia Costa	sofia.costa@campus.fr	Computer Science	student	open sourcecloud computingvolunteering	7	Edit
Fatou Diallo	fatou.diallo@campus.fr	Data Engineering	student	data scienceartificial intelligencesports	7	Edit
Chloe Dubois	chloe.dubois@campus.fr	Design	student	designphotography	5	Edit
Nora El Amrani	nora.elamrani@campus.fr	Business Administration	student	entrepreneurshipdesign	9	Edit
Tom Girard	tom.girard@campus.fr	Computer Science	student	cybersecurityartificial intelligence	9	Edit
Hugo Lefevre	hugo.lefevre@campus.fr	Physics	student	roboticsmusic	0	Edit
Alice Martin	alice.martin@campus.fr	Data Engineering	student	artificial intelligencecloud computingdata science	7	Edit
Lea Moreau	lea.moreau@campus.fr	Data Engineering	student	data sciencevolunteeringmusic	8	Edit
David Nguyen	david.nguyen@campus.fr	Data Engineering	professor	data scienceartificial intelligence	7	Edit
Claire Petit	claire.petit@campus.fr	Computer Science	staff	open sourcenetworkingcloud computing	5	Edit

User Detail

Full participation history for one user, built with a `$lookup` into events whose sub-pipeline runs `$unwind` on registrations and then `$match` on the current `userId` - this is the `$lookup` required by the brief. The "upcoming" and "past" counters are derived in Python from that same result.

Campus Event Manager

DashboardEventsUsersAnalytics

Lucas Bernard

EditDelete

lucas.bernard@campus.fr - Mechanical Engineering - student

Interests

roboticssports

4 Upcoming

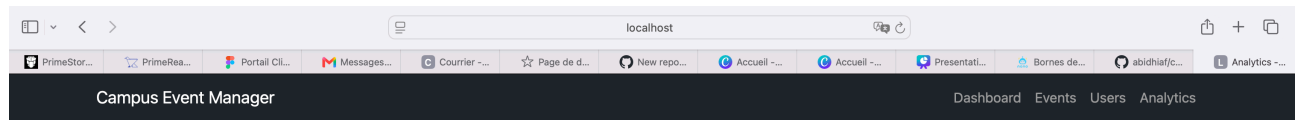
3 Past

Event participation (7)

Event	Category	Date	Status
Intro to Python for Data Science	Workshop	15 Aug 2026, 09:00	Confirmed
Cloud Architecture Fundamentals	Talk	22 Aug 2026, 09:00	Confirmed
Cybersecurity Career Night	Career Fair	09 Sep 2026, 09:00	Waiting
Student Robotics Meetup	Meetup	12 Sep 2026, 09:00	Waiting
Entrepreneurship 101	Talk	08 Oct 2026, 09:00	Confirmed
UX Research Workshop	Workshop	04 Nov 2026, 09:00	Confirmed
Security CTF Qualifiers	Hackathon	10 Nov 2026, 09:00	Confirmed

Analytics

The six required analyses (registrations by category, top 5 most popular events, users with no registration, events above average occupancy, most used tags, events by month), plus a "Key Insights" panel at the top that turns those six results into a handful of plain-English takeaways - the busiest month, the leading category, and so on - computed live from the same pipelines rather than written by hand.



Analytics

Key insights

- "Workshop" leads engagement: 23 confirmed registrations across 5 events — 33.8% of all confirmed sign-ups.
- "career" is the most common tag, used on 6 events.
- 2026-09 is the busiest month: 5 events and 31 registrations.
- 5 event(s) run above the 15.7% average occupancy rate, led by "Student Robotics Meetup" at 90.0%.
- 1 user(s) have not registered for a single event yet — worth a reminder email.

A. Registrations by category

Category	Number of events	Confirmed registrations
Workshop	5	23
Meetup	3	15
Talk	5	15
Hackathon	3	8
Career Fair	2	7

B. Top 5 most popular events

Title	Category	Capacity	Confirmed	Occupancy
Student Robotics Meetup	Meetup	10	9	90.0%
Intro to Python for Data Science	Workshop	25	7	28.0%

6. Two Aggregation Pipelines, in Detail

Of the six analyses on the Analytics page, two go well beyond a simple group/sort and deserve a closer look.

Users with no registration (Analysis C)

The idea is to start from users and, for each one, look up whatever registrations they might have in events through a `$lookup` with a sub-pipeline - necessary here because the match isn't on a direct field but on an element buried inside an array:

```
db.users.aggregate([
  { $lookup: { from: "events", let: { uid: "$_id" },
    pipeline: [
      { $unwind: "$registrations" },
      { $match: { $expr: { $eq: ["$registrations.userId", "$$uid"] } } }
    ], as: "regs" } },
  { $match: { regs: { $size: 0 } } } ]])
```

The `$lookup`'s sub-pipeline first unwinds every event's registrations array, then keeps only the entries whose `userId` matches the current user (`$$uid`, passed in through `let`). The resulting `regs` field is therefore

either empty or full of the events that user appears in. The final `$match` keeps only the documents where `regs` is an empty array - in other words, users who are never registered anywhere.

Events above average occupancy (Analysis D)

This is the most involved pipeline in the project, because it needs a global average computed across every event and then a per-event filter against that same average - two operations working at different granularities inside one pipeline. I used `$facet` to compute both branches in parallel from the same set of documents:

```
{ $addFields: { confirmedCount: ..., occupancyPct: ... } },
{ $facet: {
  events: [ { $project: {...} } ],
  stats: [ { $group: { _id: null, avgOccupancy: { $avg: "$occupancyPct" } } } ]
} },
{ $unwind: "$stats" },
{ $project: { avgOccupancy: "$stats.avgOccupancy",
  events: { $filter: { input: "$events", as: "e",      cond: { $gt: ["$$e.occupancyPct",
"$stats.avgOccupancy"] } } } } }
```

The stats branch of the `$facet` computes the global average with `$avg`, while the events branch keeps every event individually. Since `$facet` returns a single document with both branches side by side, a `$filter` afterwards is enough to keep only the events whose occupancy percentage is above the average computed right next to it. This is the pipeline that actually exercises `$avg`, one of the operations the brief explicitly asks for.

7. Conclusion

The most time-consuming part wasn't writing pipelines, it was keeping the seed data honest. While testing the app in the browser I noticed my first seed script could register the same user twice for the same event, whenever I artificially topped an event up to its capacity to test the "Full" badge - the duplicate showed up right there in the participant list. I had to rewrite the generator to track, event by event, which users were already registered before adding more. It was a good reminder that the application-level check in `register_user` (which refuses a duplicate before any `$addToSet`) is worthless if the starting dataset isn't clean to begin with.

The other thing that took a while to get right was combining a `$jsonSchema` validator with a cross-field rule (endDate can't be before startDate). `$jsonSchema` alone has no way to compare two fields against each other, so the validator had to be wrapped in an `$and` alongside a separate `$expr` condition. Once I understood that schema validation and query expressions can live side by side inside the same validator, a few other things clicked into place too.

As an accepted limitation, there's no authentication (out of scope per the brief): anyone can register or cancel any user from an event's page. In a real deployment I'd also want pagination on the Events and Users pages once the dataset grows past a handful of screens - with 15 users and 18 events it wouldn't do anything useful right now, but the indexes on `startDate`, `category`, and the two compound indexes (`category+startDate`, `department+role`) are already there to support it later. I did add a small `pytest` suite (`tests/test_validation.py`, 16 tests) covering the form-validation rules, wired into GitHub Actions so

it runs on every push - not full coverage of the application, but enough to catch a regression in the business rules (capacity, dates, email) before it reaches the database.

Ahmed ABIDHIAF